NarraTer: orquestração de agentes de IA de fornecedores diferentes pelo terminal

Andamento do Trabalho de Conclusão de Curso

Gabriel Cortez

12 de novembro de 2026

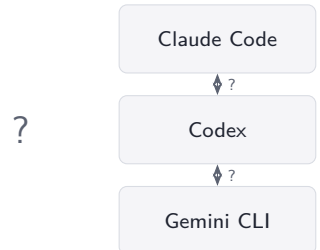
Roteiro

1. O problema: agentes de código não conversam entre si
2. O NarraTer e a tese central (o terminal como API universal)
3. Como o projeto produz evidência
4. Estudo 1: saber quando um agente terminou
5. Estudo 2: benchmark de orquestração entre sete versões
6. Limitações, próximos passos e perguntas ao orientador

O problema

Agentes de código vivem em terminais isolados

- Os agentes de programação mais usados hoje são **programas de terminal**: Claude Code, Codex, Gemini CLI, OpenCode.
- Cada um é de um fornecedor, com sua interface e seus formatos internos.
- Eles não conversam entre si. Quem passa contexto de um para outro é a pessoa, copiando e colando.
- Quando um fornecedor oferece times de agentes, só funciona **dentro do próprio produto**.



Pergunta de pesquisa

Como fazer agentes de IA de **fornecedores diferentes** trabalharem juntos **sem exigir que eles cooperem** com um protocolo próprio, e quanto isso custa?

Desdobrada em três perguntas que dá para medir:

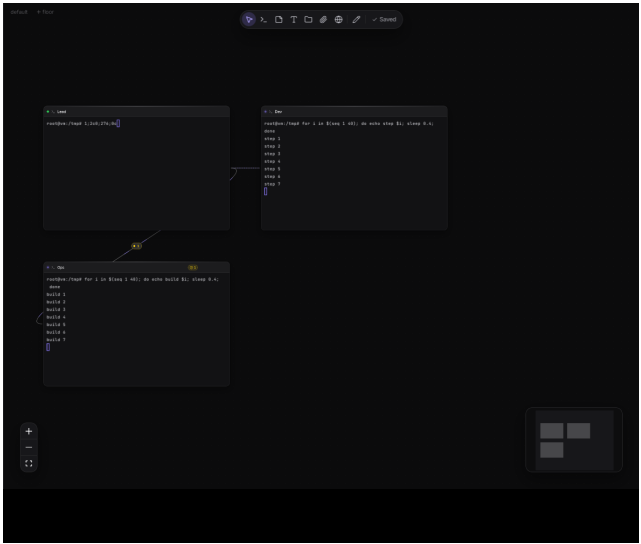
Q1 Como saber, só olhando o terminal, que um agente **terminou** o turno?

Q2 Como obter a **resposta inteira** do agente, e não um pedaço da tela?

Q3 Quanto um protocolo agnóstico custa em **tokens, tempo e trabalho feito**, comparado a um protocolo cooperativo preso a um só fornecedor?

O NarraTer

O NarraTer: um canvas de terminais



- Aplicativo para Linux (Tauri 2: Rust + React).
- Cada **nó** é um terminal real (um PTY).
- Cada **aresta** é uma permissão: quem pode falar com quem.
- Os agentes conversam por uma CLI, `narrater`, que o app instala.
- O estado de cada terminal (trabalhando, pronto, precisa de você) vem da tela.

Exemplo: um líder delega a um desenvolvedor

O Narrador (líder) roda no seu terminal:

```
$ narrater ask DevA "Implemente a
    conversão de moedas e rode os testes.
    Responda com o resultado."
```

E recebe de volta a resposta final do DevA:

```
Implementei a funcionalidade multimoeda
e os testes passam (43 testes, OK). As
alterações estão em pedidos/cambio.py,
modelo.py ...
```

O DevA não instalou nada nem conhece protocolo: o pedido foi digitado no terminal dele.



A tese central: o terminal é a API universal (decisão D01)

Julho: modelo cooperativo

- Servidor MCP, *prompt* de sistema extra, *hook* de parada.
- O alvo responde por ID com uma ferramenta própria.
- Só funcionava com o Claude Code. Um alvo que não conhece o protocolo trava o pedido até o *timeout*.

Outubro: modelo agnóstico

- O estado do turno é lido da **tela renderizada** (emulador vt100 sem interface).
- O pedido é digitado quando o alvo está ocioso.
- Instruções chegam por arquivo (`CLAUDE.md`, `AGENTS.md`), que toda CLI lê.
- Qualquer programa que roda num terminal participa.

Regra derivada (D05/D06)

Cooperação do agente só entra como **opcional que melhora o resultado**. Se o agente não cooperar, tudo continua funcionando pela tela.

Trabalhos relacionados estudados

Projeto	Como sabe que o agente terminou	Como obtém a resposta	Exige cooperação?
Maestri (macOS)	tela	tela	não
Orca	*hooks* em ∼25 CLIs + regras de tela em JSON + título da janela	ferramenta `worker-read`, blocos de 1.200 caracteres	sim
CLI Agent Orchestrator (AWS)	expressões regulares por CLI sobre a tela	tela	não
Agent Teams (Claude Code)	evento interno do programa	mensagem empurrada	só Claude
NarraTer	tela + título + *transcript* quando existe	*transcript*, ou tela	**não**

Estudos em `docs/pesquisa/` (código-fonte lido em versões e *commits* registrados). Falta a revisão de literatura acadêmica (ver "Próximos passos").

Como o projeto produz evidência

Registro de decisões com evidência classificada (D09)

Cada decisão com alternativa real vira uma entrada em `docs/decisoes.md` (21 até agora), com seis campos:

- contexto, decisão;
- alternativas recusadas e por quê;
- evidência **rotulada**: [medido] [observado] [citado] [inferido]
- consequência;
- **condição que derrubaria a decisão**.

O arquivo só cresce: uma decisão revista ganha entrada nova que cita a antiga.

```
## D20 · A resposta do ask e o fim do turno vêm
##       do transcript do agente quando ele existe

Alternativas consideradas.
- Scrollback do emulador: o Claude Code roda em
  tela alternativa, o histórico é vazio [medido]
- Usar só o end_turn: o bloco de pensamento traz
  end_turn 15 s antes do texto [observado]

Evidência.
- [medido] acha a resposta certa em 126 de 126
  pedidos das rodadas C a G
- [medido] asks prontos antes da hora:
  D 4 de 27, E 12 de 31, F e G 0 de 40

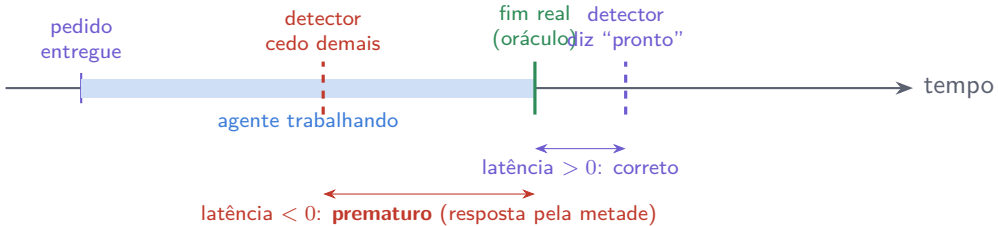
Revisar se: o Claude Code mudar o formato do
transcript; o benchmark voltar a mostrar asks
antes da hora ou respostas trocadas.
```

Regra de método: medir contra o programa real (D12)

- Toda regra que lê a tela de um agente é testada contra o **programa real**, nunca contra uma tela lembrada.
- Cada execução medida é gravada (asciicast) e vira **teste de regressão**: `tests/transcripts/` tem gravações reais do Claude Code (resposta curta, longa, ferramenta silenciosa, diálogo de permissão...).
- A verdade vem de um **oráculo independente da tela**, ligado só no avaliador:
  - shell: o próprio comando grava um *timestamp* ao terminar;
  - Claude Code: *hooks* `Stop` e `PermissionRequest`, configurados só na medição.
- O app continua sem *hooks*: o oráculo mede, não participa (D01 preservada).

Estudo 1: saber quando um agente terminou (Q1)

Método: latência contra o oráculo



- Detector = o próprio código do app (`screen.rs`), no mesmo ciclo de 200 ms.
- Cenários: shell silencioso, com saída contínua, rajada + silêncio (direto e dentro do tmux); Claude Code com resposta curta, ferramenta silenciosa, resposta longa e pedido de permissão.
- Ferramenta: `cargo run -release -example turn_eval - -reps N [-claude]`

Resultados: antes e depois das correções (D10, D11)

	Antes		Depois	
Cenário	corretos/prem. (n)	latência	corretos/prem. (n)	latência
shell direto (3 cenários)	9 / 0 (9)	0,6–0,8 s	30 / 0 (30)	0,6–0,8 s
tmux, saída contínua	3 / 0 (3)	0,76 s	10 / 0 (10)	0,75–0,78 s
tmux, comando silencioso	0 / 3 (3)	−3,2 s	10 / 0 (10)	0,6–0,8 s
tmux, rajada + silêncio	0 / 3 (3)	−2,2 s	10 / 0 (10)	0,8 s
Claude, turnos normais	9 / 0 (9)	mediana 1,36 s	15 / 0 (15)	mediana 0,55 s
Claude, permissão	nunca detectado	—	3 / 0 (3)	0,1–0,2 s

[medido] Claude, turnos normais: antes n=9, depois n=15, faixas sem sobreposição (Mann–Whitney exato, $p \approx 1{,}5 \times 10^{-6}$). tmux silencioso: 6/6 prematuros antes, 0/20 depois (Fisher exato, $p \approx 4{,}3 \times 10^{-6}$).

Ressalvas: uma máquina, uma versão do Claude Code (2.1.299, modelo Haiku); Codex e Gemini não medidos.

Exemplo de achado: o que estava errado no tmux

[observado] Dentro do tmux, o processo em primeiro plano do PTY é o *cliente tmux*, não o comando. Durante um `sleep 4`, a tela fica parada e o leitor concluía "terminou".

Primeira correção: ler o primeiro plano do painel tmux pelo `/proc`. Ainda deu 4 de 15 "prematuros" de 2–3 ms: a leitura caía entre o fim do comando e o redesenho do *prompt*.

Correção final (D10): tratar "há um comando em primeiro plano" como atividade de tela. [medido] 0 prematuros em 20.

[observado] Outro achado: com o diálogo de permissão aberto, o Claude Code às vezes pisca um indicador a cada 0,6 s (289 piscadas em 180 s), e a tela nunca fica parada. O leitor ficava preso até o *timeout*.

Correção (D11): o **título da janela** (que o Claude Code muda conforme o estado) entra no leitor. [medido] detectado em 0,1–0,2 s.

[medido] O oráculo também foi escolhido por medição: o *hook* `Notification` chega 6 s atrasado; o `PermissionRequest` chega na hora.

Estudo 2: benchmark de orquestração (Q2, Q3)

Desenho experimental (D19)

Tarefa: um repositório Python inventado (`pedidos`, ∼300 linhas) com regras de negócio próprias.

- implementar pedidos em várias moedas (atravessa 3 módulos);
- corrigir 2 defeitos descritos só como reclamação de cliente;
- manter e escrever testes.

Julgamento: 31 **testes escondidos** que os agentes nunca veem, em 4 níveis. Solução de referência: 31/31. Repositório inicial: 5/31.

Uma calculadora foi recusada: o modelo sabe o exercício de cor.

Equipe fixa:

- Narrador (líder, não edita): Claude Opus 5.5
- DevA: Claude Sonnet 5.5
- DevB: Claude Haiku 5.5

Controles:

- toda rodada começa **zerada** (HOME, sessões, tmux, pasta);
- mesma versão do Claude Code em todas;
- **ordem contrabalançada** (o *cache* do servidor dura 1 h);
- 3 rodadas por versão;
- métricas tiradas das transcrições, não do que o agente diz.

As versões comparadas: cada uma é um *commit* real

	Commit	O que muda
A	`e708d7b`	Cooperativo (julho): MCP, *prompt* de sistema, *hook*. Só Claude. O dev envia o relatório inteiro.
B	`16cee0e`	Agnóstico (D01): CLI + *skill*; o `ask` devolve a tela do alvo.
C	`95fb11a`	`ask` assíncrono e durável.
D	`db3d535`	Leitor de estado novo (Estudo 1) e *statusline*.
E	`8e2e5fa`	A resposta vem do *transcript* do agente, não da tela. **E-sl0**: E sem *statusline* (ablação).
F	`26e3c06`	O *transcript* também decide se o turno acabou.
G	`c6ba63e`	Cada `ask` acha o próprio turno (corrige pedidos paralelos).

A–D mediram a história do projeto. E–G foram criadas **a partir do diagnóstico** de A–D, cada uma corrigindo uma falha vista nas rodadas da anterior.

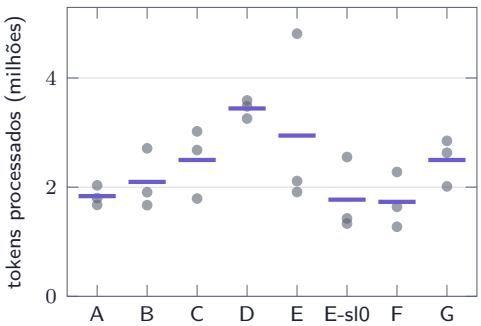
Resultado 1: a qualidade bateu no teto

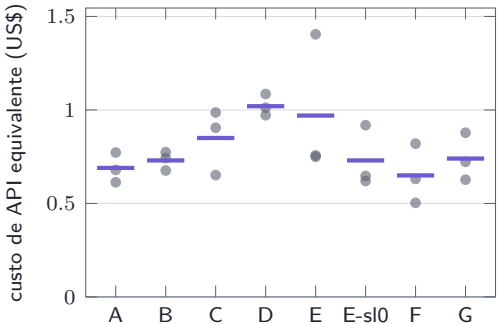
24/24

rodadas passaram em **31 de 31** testes escondidos

- [medido] Todas as versões, todas as rodadas, nota máxima. Os testes da própria equipe também passaram (39 a 62 testes).
- O líder nunca editou um arquivo: o papel foi respeitado.
- **Consequência para o método**: esta tarefa mede **custo**, não capacidade. Para comparar qualidade, é preciso uma tarefa mais difícil.

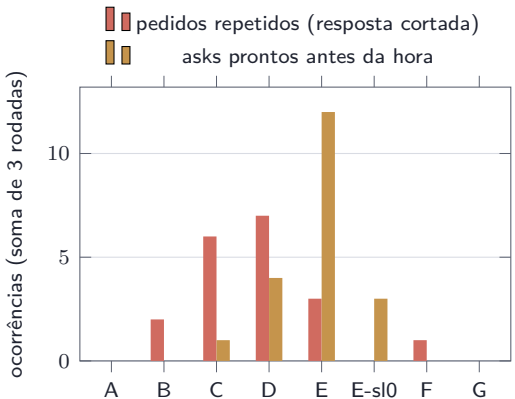
Resultado 2: custo por versão





Cada ponto é uma rodada; o traço é a média. [medido] De A até D o custo **subiu** (1,8 → 3,4 milhões de tokens; US$ 0,69 → 1,02). Com F e G ele voltou ao nível da A (US$ 0,65 e 0,74), **sem** prender o NarraTer a um fornecedor.

Por que D custou mais? O mecanismo encontrado



"Antes da hora" só é medível a partir da C.

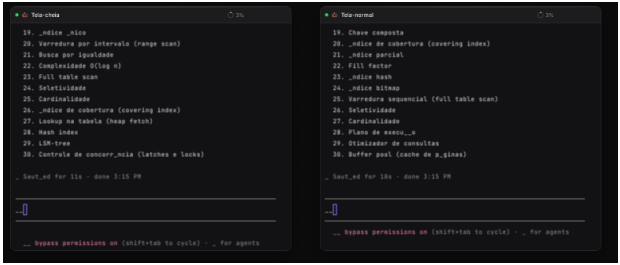
[medido] A latência do protocolo é de segundos (entrega 0,2–0,5 s; detecção 1,6–3,1 s). A duração cresce em **minutos**: o que cresce é o número de pedidos do líder (3–5 na A, 7–10 na D).

Duas falhas fazem o líder pedir de novo:

1. **resposta cortada**: a tela mostra ∼13 linhas, e o relatório do dev é maior;
2. **ask antes da hora**: o leitor dava o turno por encerrado 4–5 s após a entrega, antes de o agente começar.

[medido] Na rodada E2, 8 de 16 asks voltaram cedo; o líder deixou de confiar no `ask`: 14 pedidos e 10 minutos.

Por que a tela não basta para o Claude Code



O mesmo agente em tela cheia e normal: o terminal só guarda o que está visível.

[medido] O Claude Code desenha numa **tela alternativa**: não sobra histórico de rolagem no emulador. A ideia inicial (ler o *scrollback*) não funcionaria.

Solução (D20): o próprio Claude Code grava um *transcript* em JSON Lines no disco. O NarraTer **lê** esse arquivo, sem pedir nada ao agente. Se o arquivo não existir, a tela continua valendo.

Exemplo real: o `end_turn` que mente

Trecho do *transcript* da rodada E2 (hoje é um teste em `tests/transcripts/`), resumido:

```
15:40:32.7  user       "[narrater from Narrador]: Projeto: .../pedidos. NÃO altere nada..."
15:40:35.3  assistant  tool_use  Bash
15:40:37.3  user       tool_result
15:40:47.7  assistant  thinking            stop_reason = end_turn     <- parece o fim
15:41:03.1  assistant  text "Não alterei nada. Li REGRAS.md, ..."  stop_reason = end_turn
15:41:03.2  system     turn_duration                                    <- fim de verdade
```

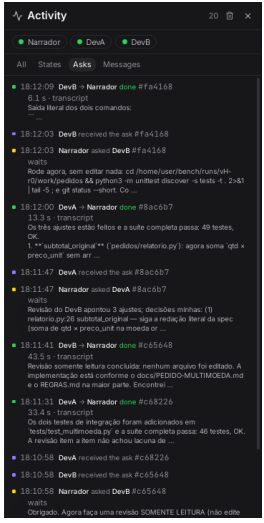
- [observado] O Claude Code grava cada bloco da mensagem como um registro, e todos levam o `stop_reason`. O pensamento já diz `end_turn` **15 s antes** de a resposta existir.
- O fim confiável é o registro `system/turn_duration`. A versão F passou a esperar por ele.
- É formato interno do fornecedor e pode mudar: por isso a tela continua como reserva e o benchmark é o alarme.

Resultado 3: as falhas do protocolo foram a zero

	A (cooperativo)	D	E	F	G
Asks prontos antes da hora	—	4/27	12/31	**0/16**	**0/24**
Pedidos repetidos (por rodada)	0, 0, 0	1, 5, 1	0, 2, 1	0, 1, 0	**0, 0, 0**
Pedidos do líder aos devs	3–5	7–10	5–14	3–7	3–6
Tokens processados (milhões)	1,84	3,44	2,94	1,73	2,50
Custo médio (US$)	0,69	1,02	0,97	0,65	0,74
Prende a um fornecedor?	sim	não	não	não	não

- [medido] D contra G: sem sobreposição em tokens, custo e pedidos (Mann–Whitney exato, U = 9, $p = 0{,}10$, o menor p possível com 3 contra 3).
- [medido] O leitor de *transcript* acha a resposta certa em 126 de 126 pedidos reais das rodadas C–G.
- [inferido] O que ainda separa G de A é o ritmo dos agentes, não a coordenação: na G3, 243 s foram o DevB (Haiku) trabalhando sem parar.

Verificabilidade: o painel Activity (D21)



- Cada estado, pedido, entrega e resposta é gravado **pelo backend**, no momento em que acontece (`activity.jsonl`).
- Nenhum agente escreve nesse registro: é o fato, não a palavra da IA.
- Cada resposta mostra a duração e a **origem** (transcript ou tela).
- Serve para auditar uma orquestração depois, e para explicar falhas como o ask antes da hora.

Limitações e próximos passos

Ameaças à validade

- **Amostra pequena**: 3 rodadas por versão. O menor p possível é 0,10; nenhuma diferença é significativa a 5%. São indicações fortes, não provas.
- **Teto de qualidade**: 31/31 em tudo. A tarefa só mede custo.
- **Um fornecedor só**: todos os agentes medidos são Claude. A vantagem central da tese (falar com Codex, Gemini…) **ainda não foi medida**.
- **Correções guiadas pelos dados**: E–G nasceram das falhas de A–D, na mesma tarefa. Rodadas novas confirmam o sumiço das falhas, mas há risco de sobreajuste à tarefa.
- **E–G sem intercalar com A–D**: rodaram depois, na mesma tarde.
- **Ambiente único**: um contêiner, uma versão do Claude Code; as interfaces mudam entre versões.
- **Heurísticas de contagem**: "resposta cortada" detectada por palavras e conferida à mão.

Próximos passos

	O que	Responde a
1	**Agentes de outros fornecedores** no benchmark (Codex, Gemini CLI) e leitor de *transcript* do Codex	Q1, Q3 e a tese
2	**Mais rodadas** (≥ 8 por versão), intercaladas, para separar versões vizinhas com significância	todas
3	**Tarefa sem teto**, em que a qualidade varie entre versões	Q3
4	**Revisão de literatura** acadêmica sobre sistemas multiagente com LLMs e *benchmarks* de engenharia de software	enquadramento
5	Tarefas com dependências e pega automática (O8), medidas no mesmo benchmark	Q3

Contribuições até aqui

1. Um **protocolo de orquestração agnóstico**, implementado e usado, em que nenhum agente precisa cooperar.
2. Um **método de medição** do leitor de estado contra um oráculo independente, com gravações reais transformadas em testes de regressão.
3. Um **benchmark reprodutível** de orquestração (tarefa inventada, testes escondidos, rodadas zeradas, ordem contrabalançada), aplicado a sete versões reais do sistema.
4. Evidência [medido] de que o protocolo agnóstico chegou ao **mesmo custo** do cooperativo (US$ 0,65–0,74 contra 0,69) depois de identificar e corrigir dois mecanismos de falha.
5. Um registro de 21 decisões com alternativas, evidência classificada e condição de revisão.

Perguntas para o orientador

- O **recorte** está bom? Ferramenta + método de medição, ou o foco deveria ser só o estudo experimental?
- Para o texto final, quantas rodadas por versão e que nível de significância são esperados?
- A validação com **outros fornecedores** é obrigatória para sustentar a tese, ou pode ficar como trabalho futuro?
- Que **literatura** deve servir de base (sistemas multiagente com LLMs, *benchmarks* de agentes de código)?
- Como declarar o uso de um agente de IA no desenvolvimento e na análise?

Obrigado

Código, dados brutos, relatório completo e como reproduzir:

`docs/relatorios/orquestracao-versoes.pdf` `bench/orquestracao/` `docs/decisoes.md`

Apêndice: como reproduzir

Leitor de estado (Estudo 1):

```
cd src-tauri && cargo run --release --example turn_eval -- --reps 5 --claude
cd src-tauri && cargo run --example turn_replay -- GRAVACAO.cast --at 8
cd src-tauri && cargo test turn_transcripts
```

Benchmark (Estudo 2):

```
git worktree add --detach $BENCH/vX <commit>      # uma por versão
$BENCH/build.sh A B C D E F G                       # compila cada versão
Xvfb :77 -screen 0 1600x1000x24 &
cd bench/orquestracao && ./rodar_todas.sh           # rodadas zeradas, ordem contrabalanç
    ada
python3 analisar.py $BENCH/runs/v?-r[1-9] && python3 agregar.py
```

Métricas: tokens processados = entrada + *cache* lido + *cache* criado, por chamada à API, tirados das transcrições. Custo = preço de API equivalente (a conta usada é de assinatura).